

Database Management System

MODULE 1

PROF. SHARANABASAPPA NOOLA



Introduction

- A **database** is a collection of related data.
- A **data** mean known facts that can be recorded and that have implicit meaning.
- For eg, consider the names, telephone numbers, and addresses of the people you know.
- These can be stored in the form of tables.
- This collection of related data with an implicit meaning is a database.
- A database can be of any size and varying complexity.
- **Data:** Facts, figures, statistics etc. having no particular meaning (e.g. 1, ABC, 19 etc).
- **Record:** Collection of related data items, e.g. in the above example the three data items had no meaning. But if we organize them in the following way, then they collectively represent meaningful information.

Roll	Name	Age
1	ABC	19

The columns of this relation are called **Fields**, **Attributes** or **Domains**. The rows are called **Tuples** or **Records**.

Table or Relation: Collection of related records.

Roll	Name	Age
1	ABC	19
2	DEF	22
3	XYZ	28

A database has the following implicit properties:

- ✓ A database represents some aspect of the real world, sometimes called the mini world or the universe of discourse (UoD). Changes to the mini world are reflected in the database.
- ✓ A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.
- ✓ A database is designed, built, and populated with data for a specific purpose. It has an intended group of users and some preconceived applications in which these users are interested.

Other properties:

- A database can be of any size and complexity.
- A database may be generated and maintained manually or it may be computerized.
- A database management system (DBMS) is a collection of programs that enables users to create and maintain a database.
- ***The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications.***
- **Defining** a database involves specifying the data types, structures, and constraints of the data to be stored in the database. The database definition or descriptive information is stored by the DBMS in the form of a database catalog or dictionary; it is called meta-data.
- **Constructing** the database is the process of storing the data on some storage medium that is controlled by the DBMS.
- **Manipulating** a database includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the miniworld, and generating reports from the data.
- **Sharing** a database allows multiple users and programs to access the database simultaneously.
- A database together with the DBMS software is referred to as a **database system**.

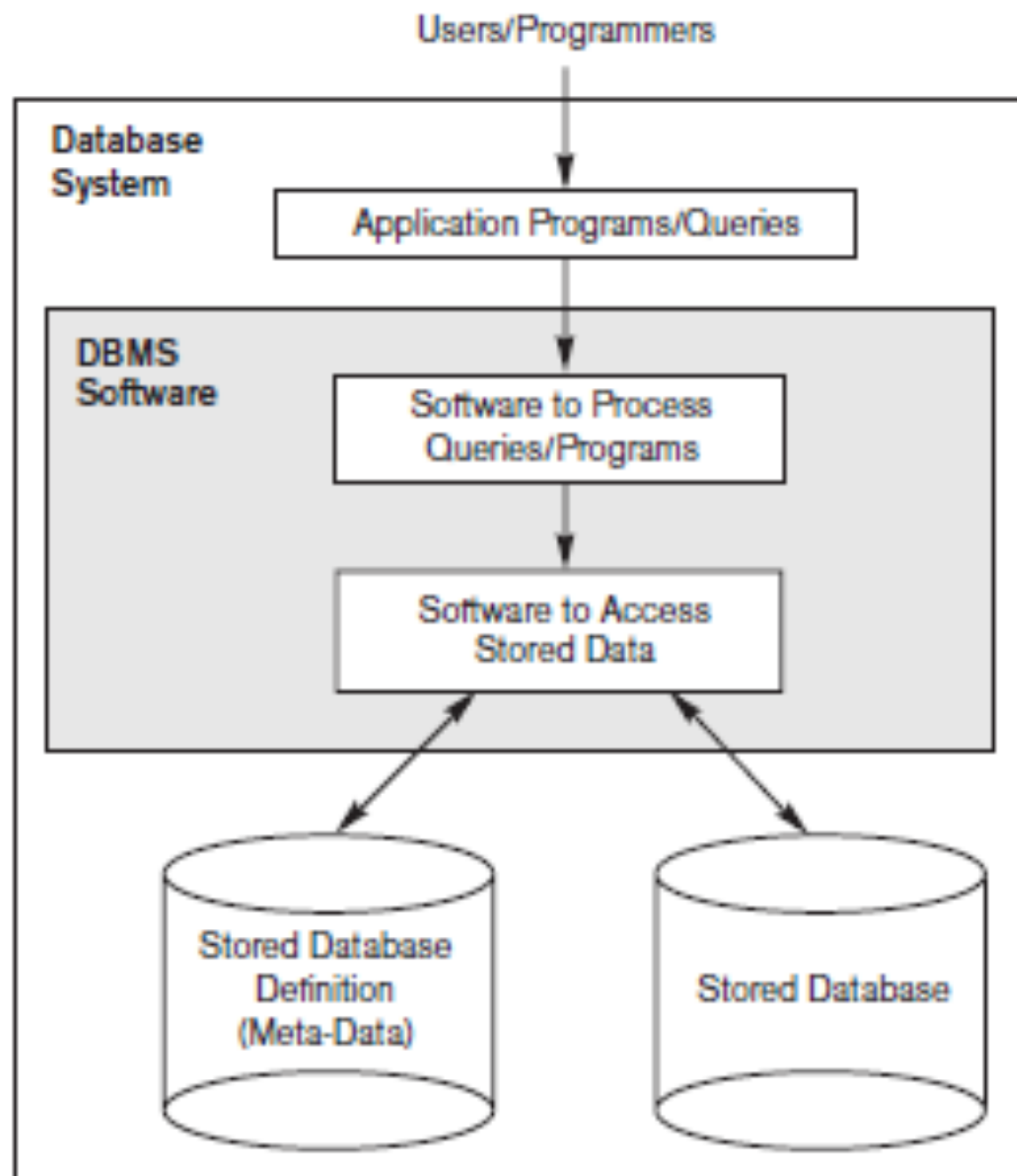


Figure 1.1
A simplified database
system environment

EXAMPLE: UNIVERSITY database

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Characteristics of the Database Approach

Database approach vs. File Processing approach.

- In the file processing approach, each department would control a collection of relevant data files and software applications to manipulate that data.
- In the database approach, a single repository maintains data that is defined once and then accessed by various users.
- In file systems, each application is free to name data elements independently. In contrast, in a database, the names or labels of data are defined once, and used repeatedly by queries, transactions, and applications.
- The main characteristics of the database approach versus the file-processing approach are the following:
 - 1. Self-Describing Nature of a Database System: -**
 - A fundamental characteristic of the database approach is that the database system contains not only the database itself but also a complete definition or description of the database structure and constraints.
 - This meta-data (i.e., data about data) is stored in the so-called system catalog, which contains a description of the structure of each file, the type and storage format of each field, and the various constraints on the data (i.e., conditions that the data must satisfy).

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

Figure 1.3

An example of a database catalog for the database in Figure 1.2.

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

2. Insulation between Programs and Data, and Data Abstraction:

■ Program-data independence:

- In a traditional file processing system, the structure of database knowledge files is embedded within the application programs, so any changes to the structure of a file may require changing all programs that access that file.
- Against this, DBMS access programs don't require such changes in most cases, so independence is achieved between them. The structure of knowledge files is stored within the DBMS catalog separately from the programs that access them. We call this property program-data independence.

■ Program-operation independence: User application programs can operate on the data by invoking operations through their names and arguments, regardless of how the operations are implemented.

■ Data Abstraction: The characteristic that allows program-data independence and program-operation independence is called data abstraction.

3. Support of Multiple Views of the Data:

- A database typically has many types of users, each of whom may require a different perspective or view of the database. A view may be a subset of the database or it may contain virtual data that is derived from the database files but is not explicitly stored.

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

Fig: view derived from the university database

4. Sharing of Data and Multiuser Transaction Processing

- A multiuser DBMS, as its name implies, must allow multiple users to access the database at the same time. The DBMS must include concurrency control software to ensure that several users trying to update the same data do so in a controlled manner so that the result of the updates is correct.
- Example: When several reservation agents try to assign a seat on an airline flight, the DBMS should ensure that each seat can be accessed by only one agent at a time for assignment to a passenger
- A transaction is an executing program or process that includes one or more database accesses, such as reading or updating of database records.
- The DBMS must enforce several transaction properties:
- The **isolation property** ensures that each transaction appears to execute in isolation from other transactions, even though hundreds of transactions may be executing concurrently.
- The **atomicity property** ensures that either all the database operations in a transaction are executed or none are executed.

➤ Users may be divided into:

- Those who actually use and control the database content, and those who design, develop and maintain database applications called **“Actors on the Scene”**.
- Those who design and develop the DBMS software and related tools, and the computer systems operators called **“Workers behind the Scene”**.

- **Actors on the Scene:**

1. **Database Administrator (DBA):** The DBA is responsible for authorizing access to the database, coordinating and monitoring its use, and acquiring software and hardware resources as needed. The DBA is accountable for problems such as security breaches and poor system response time.
2. **Database Designers:** Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data. It is the responsibility of database designers to communicate with all prospective database users in order to understand their requirements and to create a design that meets these requirements.

3. **Users:** End users are the people whose jobs require access to the database for querying, updating, and generating reports; the database primarily exists for their use.

Categories of end users:

- ❑ **Casual end users** occasionally access the database, but they may need different information each time. They are typically middle- or high-level managers or other occasional browsers.
- ❑ **Naive or parametric end users** make up a sizable portion of database end users. Their main job function revolves around constantly querying and updating the database, using standard types of queries and updates—called canned transactions—that have been carefully programmed and tested.

Examples:

- ✓ Bank customers and tellers check account balances after withdrawals/ deposits.
- ✓ Reservation agents or customers for airlines, hotels, and car rental companies check availability for a given request and make reservations.
- ❑ **Sophisticated end users** include engineers, scientists, business analysts, and others who thoroughly familiarize themselves with the facilities of the DBMS.
- ❑ **Standalone users** maintain personal databases by using ready-made program packages that provide easy-to-use menu-based or graphics-based interfaces. An example is the user of a financial software package that stores a variety of personal financial data.

4. System Analysts and Application Programmers (Software Engineers):

- **System analysts** determine the requirements of end users, especially naive and parametric end users, and develop specifications for standard canned transactions that meet these requirements.
- **Application programmers** implement these specifications as programs; then they test, debug, document, and maintain these canned transactions.

Workers behind the Scene

- **DBMS system designers and implementers** design and implement the DBMS modules and interfaces as a software package.
- **Tool developers** design and implement tools—the software packages that facilitate database modeling and design, database system design, and improved performance.
- **Operators and maintenance personnel** (system administration personnel) are responsible for the actual running and maintenance of the hardware and software environment for the database system.

Advantages of Using the DBMS Approach

- 1. Controlling Redundancy:** Data redundancy (such as tends to occur in the "file processing" approach) leads to wasted storage space, duplication of effort (when multiple copies of a datum need to be updated), and a higher likelihood of the introduction of inconsistency.
- 2. Restricting Unauthorized Access:** A DBMS should provide a security and authorization subsystem, which the DBA uses to create accounts and to specify account restrictions.
- 3. Providing Persistent Storage for Program Objects:** Object-oriented database systems make it easier for complex runtime objects (e.g., lists, trees) to be saved in secondary storage so as to survive beyond program termination and to be retrievable at a later time.
- 4. Providing Storage Structures and Search Techniques for Efficient Query Processing:** Database systems must provide capabilities for efficiently executing queries and updates. The **query processing and optimization** module of the DBMS is responsible for choosing an efficient query execution plan for each query based on the existing storage structures.

5. **Providing Backup and Recovery:** A DBMS must provide facilities for recovering from hardware or software failures. The backup and recovery subsystem of the DBMS is responsible for recovery.
6. **Providing Multiple User Interfaces:** Many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces. For example, query languages for casual users, programming language interfaces for application programmers, forms and/or command codes for parametric users, menu-driven interfaces for stand-alone users.
7. **Representing Complex Relationships Among Data:** A DBMS must have the capability to represent a variety of complex relationships among the data, to define new relationships as they arise, and to retrieve and update related data easily and efficiently.
8. **Enforcing Integrity Constraints:** Most database applications have certain integrity constraints that must hold for the data. The simplest type of integrity constraint involves specifying a data type for each data item. Referential integrity constraint, Uniqueness constraint.
9. **Permitting Inferencing and Actions Via Rules:** In a **deductive** database system, one may specify *declarative* rules that allow the database to infer new data. **Active** database systems go one step further by allowing "active rules" that can be used to initiate actions automatically.

When Not to Use a DBMS

- High initial investment and possible need for additional hardware.
- Overhead for providing generality, security, concurrency control, recovery, and integrity functions.
- If the database and applications are simple, well defined, and not expected to change.
- If there are stringent real-time requirements that may not be met because of DBMS overhead.
- If access to data by multiple users is not required.
- If the database system is not able to handle the complexity of data because of modeling limitations.
- If the database users need special operations not supported by the DBMS.

Database System Concepts and Architecture

Data Models, Schemas, and Instances

- One fundamental characteristic of the database approach is that it provides some level of data abstraction.
- **Data abstraction** generally refers to the *suppression of details of data organization and storage*, and the highlighting of the essential features for an improved understanding of data.
- A **data model**—a collection of concepts that can be used to describe the *structure of a database*—provides the necessary means to achieve this abstraction
- By structure of a database means the *data types, relationships, and constraints* that apply to the data. Most data models also include a set of basic operations for specifying retrievals and updates on the database.

Categories of Data Models

1. High-level or conceptual data models

-provide concepts that are close to the way many users perceive data. One such popular model is the entity/relationship model (ER model). The E/R model specializes in entities, relationships, and even attributes that are used by database designers.

-use concepts such as **entities, attributes, and relationships**.

- ✓ An **entity** represents a real-world object or concept, such as an employee or a project from the miniworld that is described in the database.
- ✓ An **attribute** represents some property of interest that further describes an entity, such as the employee's name or salary.
- ✓ A **relationship** among two or more entities represents an association among the entities., for example, a works-on relationship between an employee and a project.

2. Low-level or physical data models

- provide concepts that describe the details of how data is stored on the computer storage media, typically magnetic disks. The physical Data Model is used to practically implement Relational Data Model. Ultimately, all data in a database is stored physically on a secondary storage device such as discs and tapes. This is stored in the form of files, records, and certain other data structures.

- Concepts provided by low-level data models are generally meant for computer specialists, not for end users

3. Representational (or implementation) data models

- provide concepts that may be easily understood by end users but that are not too far removed from the way data is organized in computer storage. This type of data model is used to represent only the logical part of the database and does not represent the physical structure of the database. The representational data model allows us to focus primarily, on the design part of the database. A popular representational model is a Relational model.
- in between high level and low level
- In the Relational Model, we basically use tables to represent our data and the relationships between them.

Schemas

- The description of a database is called the **database schema**, which is specified during database design and is not expected to change frequently.
- A displayed schema is called a **schema diagram**.

Figure 2.1
Schema diagram for
the database in
Figure 1.2.

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

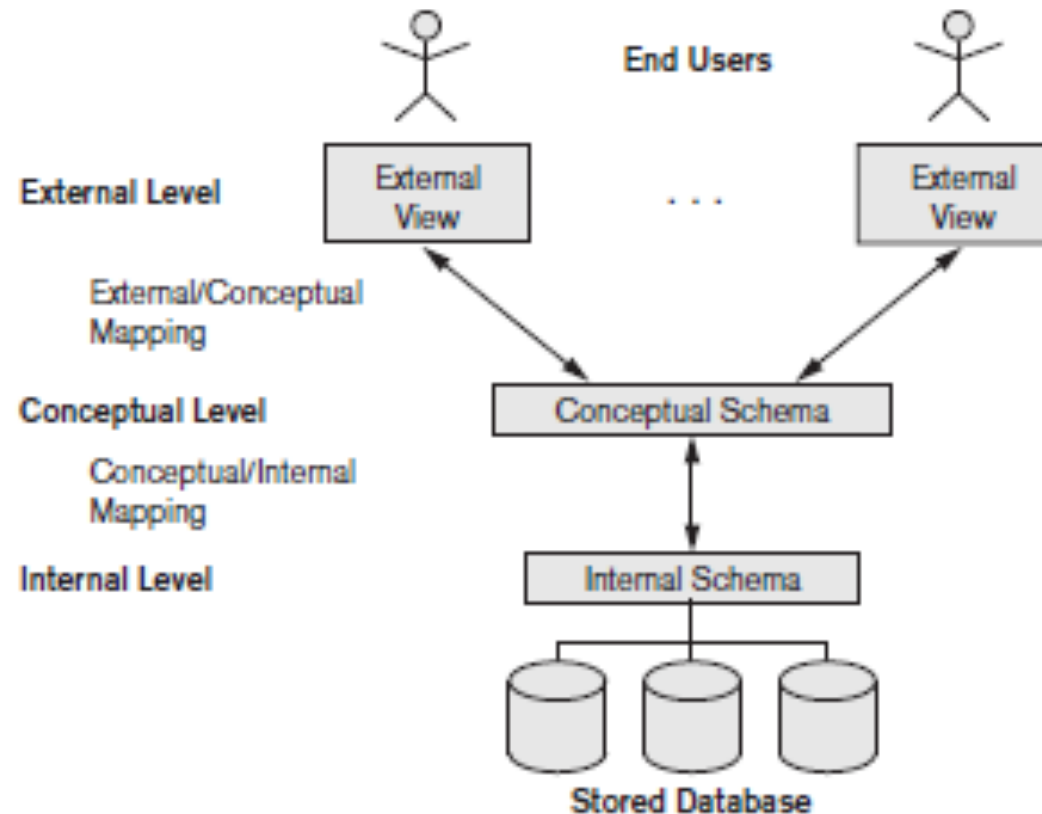
- A schema diagram displays only some aspects of a schema, such as the names of record types and data items, and some types of constraints.
- The data in the database at a particular moment in time is called a **database state or SNAPSHOT**. It is also called the **current set of occurrences or instances** in the database.
- When we define a new database, we specify its database schema only to the DBMS. At this point, the corresponding database state is the **empty state** with no data.
- We get the initial state of the database when the database is first populated or loaded with the initial data. From then on, every time an update operation is applied to the database, we get another **database state**.
- Application requirements change occasionally, which is one of the reasons why software maintenance is important. On such occasions, a change to a database's schema may be called for. An example would be to add a Date_of_Birth field/attribute to the STUDENT table. Making changes to a database schema is known as **SCHEMA EVOLUTION**

Three-Schema Architecture and Data Independence

The Three-Schema Architecture

- The goal of the three-schema architecture is to separate the user applications from the physical database.

Figure 2.2
The three-schema architecture.



In this architecture, schemas can be defined at the following three levels:

1) The **internal level** has an **internal schema**,

- -describes the physical storage structure of the database.
- -uses a physical data model and describes the complete details of data storage and access paths for the database.

2) The **conceptual level** has a **conceptual schema**,

- -describes the structure of the whole database for a community of users
- -hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints
- -representational data model is used to describe the conceptual schema when a database system is implemented
- -This implementation conceptual schema is often based on a conceptual schema design in a high-level data model.

3) The **external or view level** includes a number of **external schemas** or user views,

- -describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group.
- -As in the previous level, each external schema is typically implemented using a representational data model, possibly based on an external schema design in a high-level data model.
- The DBMS must transform a request specified on an external schema into a request against the conceptual schema, and then into a request on the internal schema for processing over the stored database.
- The processes of transforming requests and results between levels are called **mappings**

Data Independence

- **Data independence**, which can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

We can define two types of data independence:

1. Logical data independence

- capacity to change the conceptual schema without having to change external schemas or application programs
- change the conceptual schema to expand the database, to change constraints, or to reduce the database
- changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs.

2. Physical data independence

- capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well.
- by creating additional access structures—to improve the performance of retrieval or update.

Database Languages and Interfaces

➤ DBMS Languages

- Once the design of a database is completed and a DBMS is chosen to implement the database, the first step is to specify conceptual and internal schemas for the database and any mappings between the two.
- In many DBMSs where no strict separation of levels is maintained, one language, called **Data definition language (DDL)**, is used by the DBA and by database designers to define both schemas (conceptual and physical schemas).
- In DBMSs where a clear separation is maintained between the conceptual and internal levels, the **DDL** is used to specify the conceptual schema only.
- Another language, the **Storage definition language (SDL)**, is used to specify the internal schema.
- **View definition language (VDL)**, to specify user views and their mappings to the conceptual schema.
- **Data manipulation language (DML)** provides set of operations like retrieval, insertion, deletion, and modification of the data.
- DBMS packages provide an integrated feature of above languages into a single language called **Structured Query Language (SQL)**.

There are two main types of **DMLs**.

A high-level or nonprocedural DML:

- can be used on its own to specify complex database operations concisely.
- Many DBMSs allow high-level DML statements either to be entered interactively from a display monitor or terminal or to be embedded in a general-purpose programming language (C, C++, JAVA).
- can specify and retrieve many records in a single DML statement; therefore, they are called **set-at-a-time** or **set-oriented DMLs**.

A low-level or procedural DML:

- must be embedded in a general-purpose programming language (C, C++, JAVA).
- retrieves individual records or objects from the database and processes each separately.
- to retrieve and process each record from a set of records. Low-level DMLs are also called **record-at-a-time DMLs**

Whenever DML commands, whether high level or low level, are embedded in a general-purpose programming language, that language is called the **host language** and the DML is called the **data sublanguage**.

DBMS Interfaces

- ✓ **Menu-Based Interfaces** for Web Clients or Browsing. These interfaces present the user with lists of options (called menus) that lead the user through the formulation of a request.
- ✓ **Forms-Based Interfaces** displays a form to each user. Users can fill out all of the form entries to insert new data, or they can fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries.
- ✓ **Graphical User Interfaces** displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram. GUIs utilize both menus and forms. Most GUIs use a pointing device.
- ✓ **Natural Language Interfaces** accepts requests written in English or some other language and attempt to understand them.
- ✓ **Speech Input and Output** use of speech as an input query and speech as an answer to a question or result. The speech input is detected using a library of predefined words and used to set up the parameters that are supplied to the queries.
- ✓ **Interfaces for Parametric Users** such as bank tellers, often have a small set of operations that they must perform repeatedly.
- ✓ **Interfaces for the DBA.** DBA use privileged commands. These include commands for creating accounts, setting system parameters, granting account authorization, changing a schema, and reorganizing the storage structures of a database.

The Database System Environment

DBMS Component Modules

- The figure is divided into two parts.
- The **top part** of the figure refers to the various users of the database environment and their interfaces.
- The **lower part** shows the internals of the DBMS responsible for storage of data and processing of transactions.

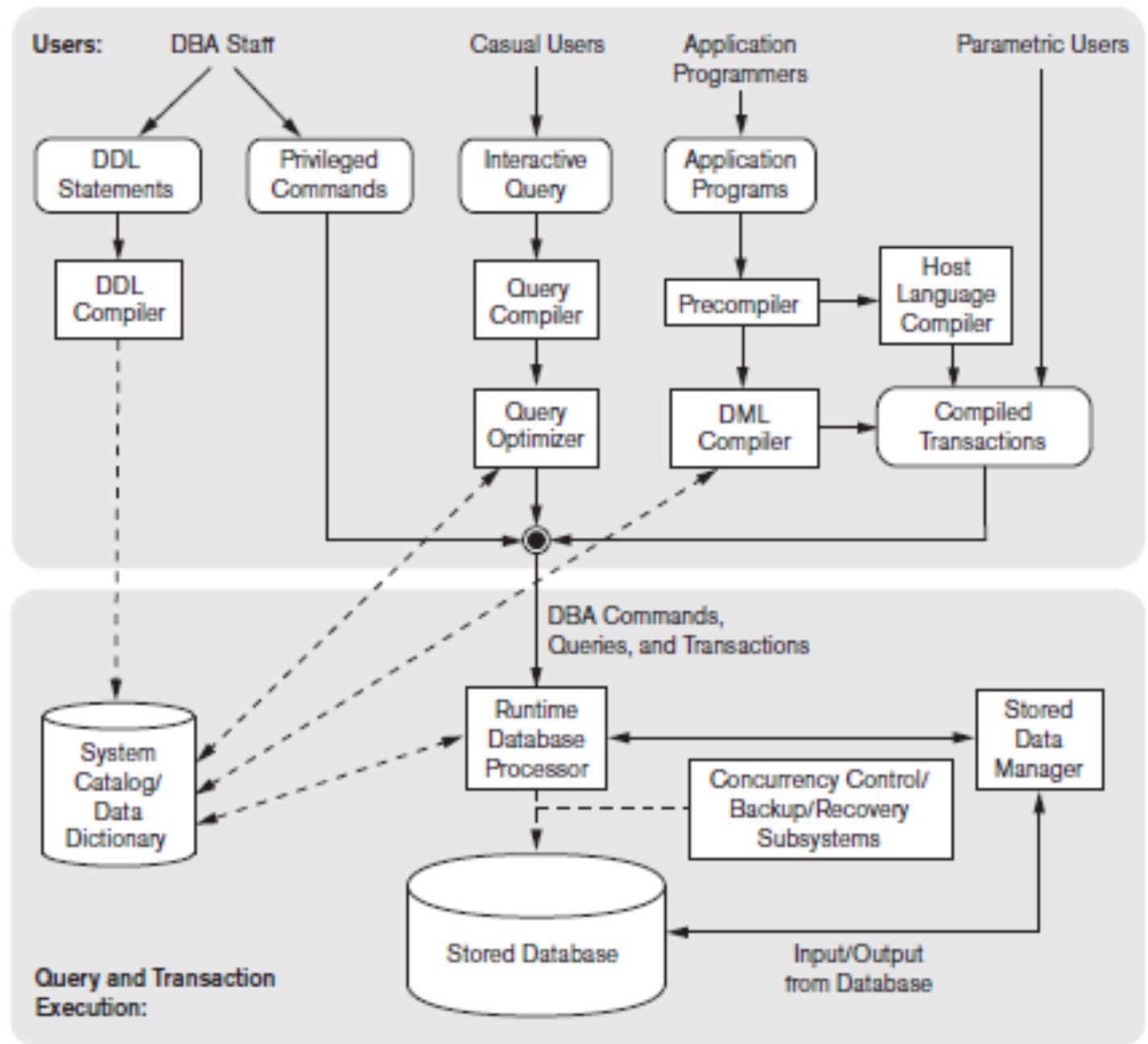


Figure 2.3

Component modules of a DBMS and their interactions.

- The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the operating system (OS), which schedules disk read/write.
- Many DBMSs have their own **buffer management** module to schedule disk read/write.
- **Stored data manager** controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.

Top half figure:

- ✓ it shows interfaces for the DBA staff, casual users who work with interactive interfaces to formulate queries
- ✓ application programmers who create programs using some host programming languages
- ✓ parametric users who do data entry work by supplying parameters to predefined transactions.
- ✓ the DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands

DBA staff:

- ✓ The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog
- ✓ The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.

Casual users:

- ✓ interact using some form of interface, which we call the interactive query interface
- ✓ queries are parsed and validated for correctness of the query syntax, the names of files and data elements, and so on by a query compiler that compiles them into an internal form
- ✓ This internal query is subjected to query optimization
- ✓ **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of correct algorithms and indexes during execution.
- ✓ It consults the system catalog for statistical and other physical information about the stored data and generates executable code that performs the necessary operations for the query and makes calls on the runtime processor

Application programmers

- ✓ write programs in host languages such as Java, C, or C++ that are submitted to a precompiler.
- ✓ precompiler extracts DML commands from an application program.
- ✓ commands are sent to the DML compiler for compilation.
- ✓ rest of the program is sent to the host language compiler.
- ✓ The object codes for the DML commands and the rest of the program are linked, forming a canned transaction.
- ✓ An example is a bank withdrawal transaction where the account number and the amount may be supplied as parameters.

In the lower part of Figure,

✓ the runtime database processor executes

(1)the privileged commands

(2)the executable query plans, and

(3)the canned transactions with runtime parameters.

✓ It works with the system catalog and may update it with statistics.

✓ It also works with the stored data manager, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory.

✓ The runtime database processor handles other aspects of data transfer, such as management of buffers concurrency control and backup and recovery systems, integrated into the working of the runtime database processor for purposes of transaction management.

Database System Utilities

DBMSs have database utilities that help the DBA manage the database system. Common utilities have the following types of functions:

✓ Loading

- used to load existing data files—such as text files or sequential files—into the database
- automatically reformats the data and stores it in the database
- for loading programs, conversion tools are available like IDMS (Computer Associates), SUPRA (Cincom), and IMAGE (HP)

✓ Backup

- creates a backup copy of the database, by dumping the entire database onto tape
- used to restore the database in case of catastrophic disk failure
- Incremental backups are also often used, to save space

✓ Database storage reorganization

- used to reorganize a set of database files into different file organizations to improve performance

✓ Performance monitoring

- monitors database usage and provides statistics to the DBA.
- Statistics used for making decisions

Tools, Application Environments, and Communications Facilities

- **CASE (Computer Aided Software Engineering)** tools are used in the design phase of database systems
- **Data dictionary** (or data repository) **system** for storing catalog information about schemas and constraints
- **Information repository** stores information such as design decisions, usage standards, application program descriptions, and user information
- **Application development environment** systems provide an environment for developing database applications, including database design, GUI development, querying and updating, and application program development
- **Communications software**, whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or personal computers
- These are connected to the database site through data communications hardware such as Internet routers, phone lines, long-haul networks, local networks, or satellite communication devices
- The integrated DBMS and data communications system is called a **DB/DC system**

Data Modeling Using the Entity–Relationship (ER) Model

Entity Types, Entity Sets, Attributes, and Keys

The ER model describes data as entities, relationships, and attributes.

Entities and Attributes

- ✓ **Entity**, which is a **thing or object in the real world** with an independent existence.
- ✓ An entity may be an
 - object with a **physical existence** (for example, a particular person, car, house, or employee) or
 - object with a **conceptual existence** (for instance, a company, a job, or a university course)
- ✓ Each entity has **attributes**—the particular **properties that describe it**. For example, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.

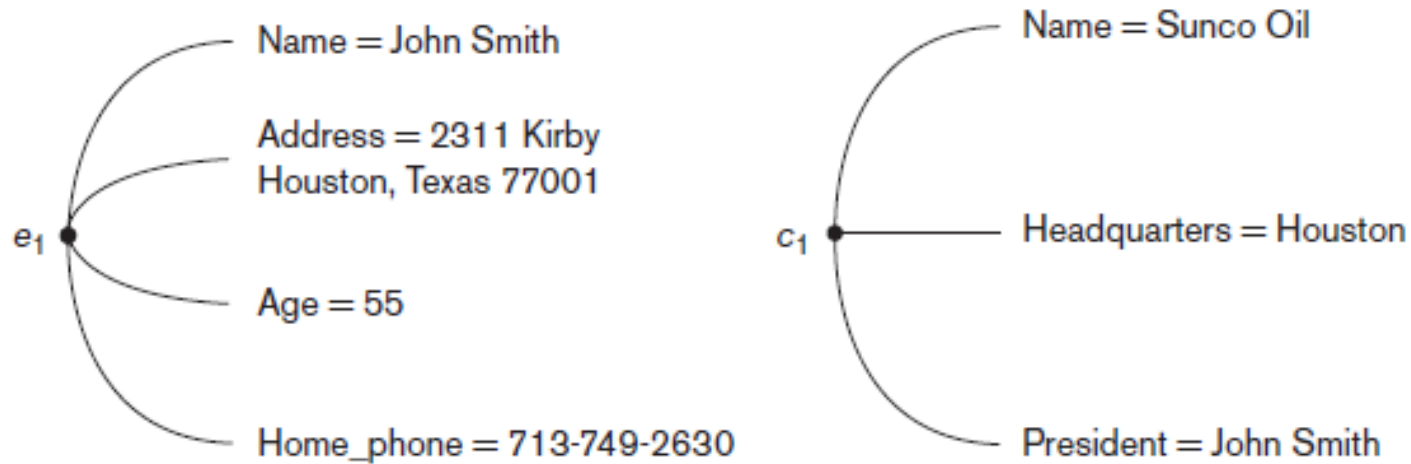


Figure 3.3
Two entities,
EMPLOYEE e_1 , and
COMPANY c_1 , and
their attributes.

- ✓ The above fig shows two entities and the values of their attributes.
- ✓ The EMPLOYEE entity e_1 has four attributes: **Name, Address, Age, and Home_phone**; their values are 'JohnSmith,' '2311 Kirby, Houston, Texas 77001', '55', and '713-749-2630', respectively.
- ✓ The COMPANY entity c_1 has three attributes: **Name, Headquarters, and President**; their values are 'Sunco Oil', 'Houston', and 'John Smith', respectively.

Types of attributes occur in the ER model

- ✓ **simple** versus **composite**
- ✓ **single valued** versus **multivalued**
- ✓ **stored** versus **derived**

❑ **Composite attributes** can be divided into smaller subparts.

For eg: the Address attribute of the EMPLOYEE entity can be subdivided into Street_address, City, State, and Zip, 3 with the values '2311 Kirby', 'Houston', 'Texas', and '77001'.

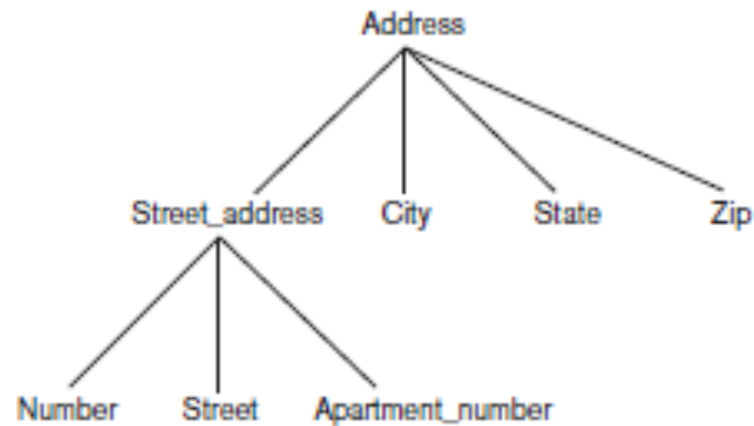


Figure 3.4
A hierarchy of
composite attributes.

❑ Attributes that are not divisible are called **simple or atomic attributes**.

For eg: attribute **Age** cannot be divided

❑ **Single-valued**

Most attributes have a single value for a particular entity.

For eg: **Age** is a single-valued attribute of a person.

❑ **Multivalued**

An entity having multiple values for that attribute .

For eg: Color attribute for a car

color={black,red}

Person's degree. degree={BE, MTech, PhD}

❑ **Stored and Derived attribute**

- ✓ Two (or more) attribute values are related—for eg: the Age and Birth_date attributes of a person
- ✓ The value of Age can be determined from the current (today's) date and the value of that person's Birth_date
- ✓ The Age attribute is hence called a **derived** attribute
- ✓ Birth_date attribute is called a **stored** attribute

❖ NULL Values

In some cases, a particular entity may not have an applicable value for an attribute. For such situations, a special value called NULL is created.

For eg - the Apartment_number attribute of an address applies only to addresses that are in apartment buildings and not to other types of residences, such as single-family homes

- College_degrees attribute applies only to people with college degrees.

❖ Complex Attributes

- ✓ composite and multivalued attributes can be nested arbitrarily.
 - ✓ arbitrary nesting by grouping components of a composite attribute between parentheses () and separating the components with commas, and by displaying multivalued attributes between braces { }.
- Such attributes are called complex attributes.
- ✓ For example, if a person can have more than one residence and each residence can have a single address and multiple phones, an attribute Address_phone for a person.

```
{Address_phone( {Phone(Area_code,Phone_number)},Address(Street_address  
(Number,Street,Apartment_number),City,State,Zip) )}
```

Figure 3.5

A complex attribute:
Address_phone.

Entity Types, Entity Sets, Keys, and Value Sets

Entity Types and Entity Sets

- ✓ **entity type** defines a collection (or set) of entities that have the same attributes
- ✓ each entity type in the database is described by its name and attributes
- ✓ below figure shows two entity types: **EMPLOYEE** and **COMPANY**, and a list of some of the attributes for each

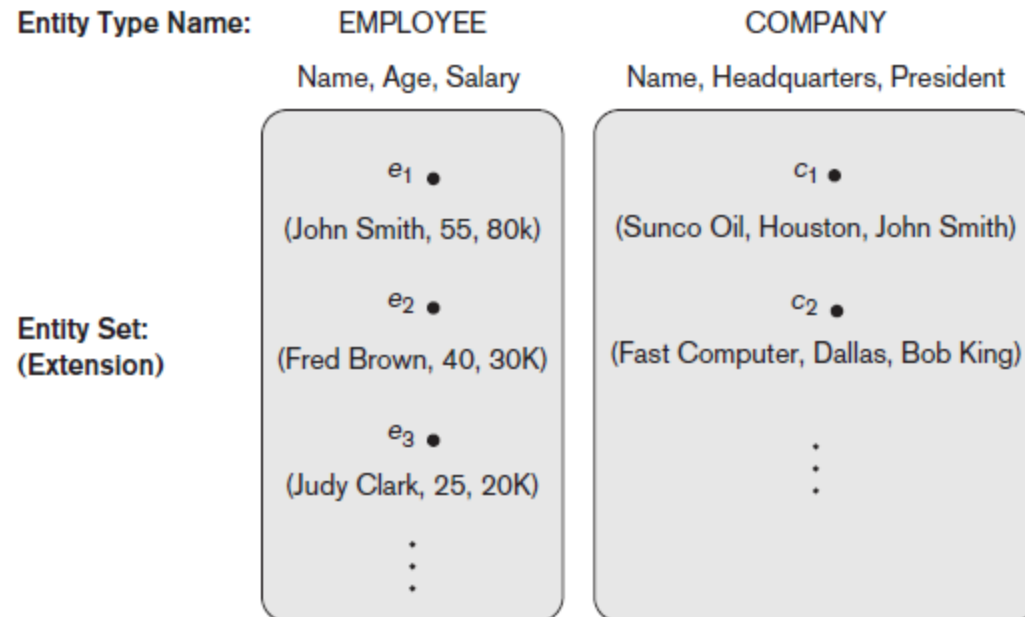
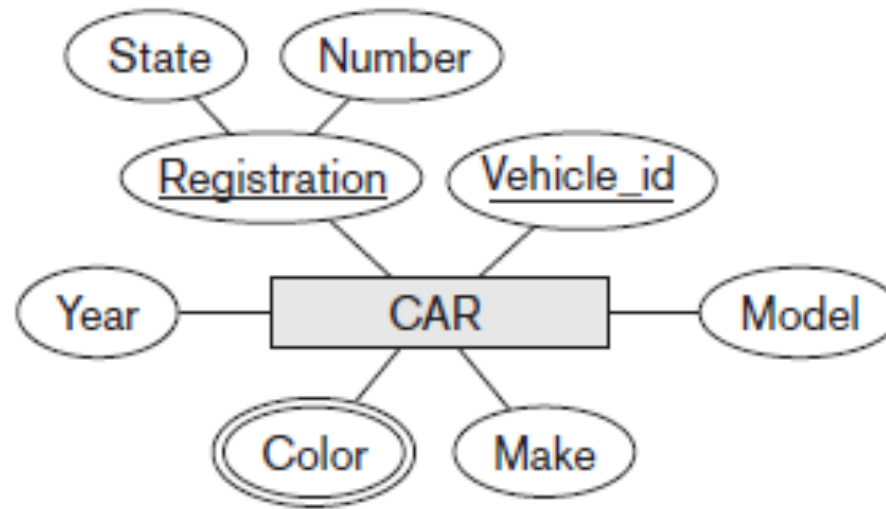


Figure 3.6

Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

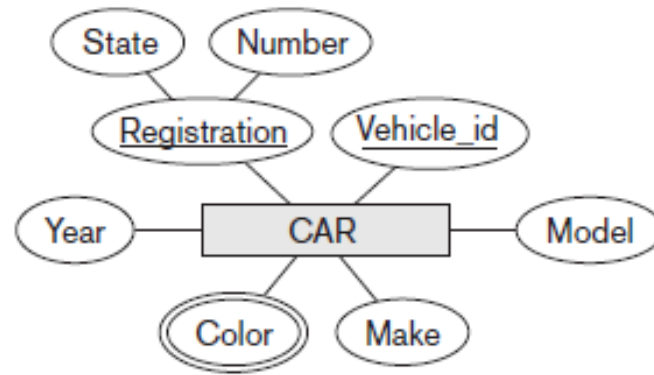
- ✓ The collection of all entities of a particular entity type in the database at any point in time is called an **entity set or entity collection**
- ✓ Entity set is usually referred to using the same name as the entity type
- ✓ An **entity type** is represented in ER diagrams as a **rectangular box**
- ✓ **Attribute names** are enclosed in **ovals** and are **attached** to their entity type by **straight lines**
- ✓ **Composite attributes** are attached to their component attributes by **straight lines**
- ✓ **Multivalued attributes** are displayed in **double ovals**
- ✓ Collection of entities of a particular entity type is grouped into an entity set, which is also called the **extension** of the entity type



Key Attributes of an Entity Type

- ✓ Important constraint on the entities of an entity type is the **key or uniqueness constraint** on attributes
- ✓ An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely
- ✓ For eg, the **Name** attribute is a key of the **COMPANY** entity type in because no two companies are allowed to have the same name
 - For the **PERSON** entity type, a typical key attribute is **Ssn**
- ✓ Specifying that an attribute is a key of an entity type means that the preceding uniqueness property must hold for every entity set of the entity type
- ✓ Hence, it is a constraint that prohibits any two entities from having the same value for the key attribute at the same time.
- ✓ Some entity types have more than one key attribute.
 - For eg, each of the Vehicle_id and Registration attributes of the entity type CAR is a key in its own right
- ✓ An entity type may also have *no key*, in which case it is called a *weak entity type*

(a)



(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR₁
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR₂
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR₃
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

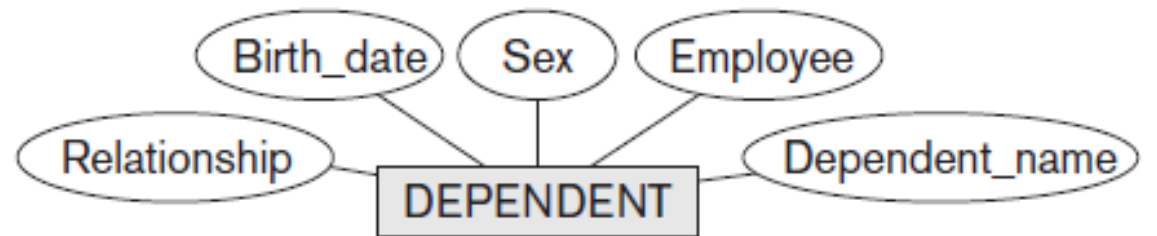
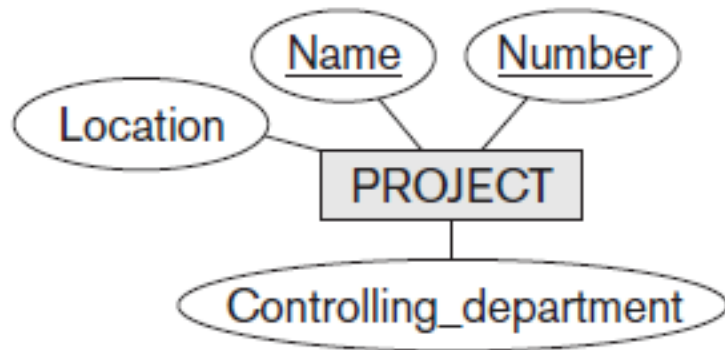
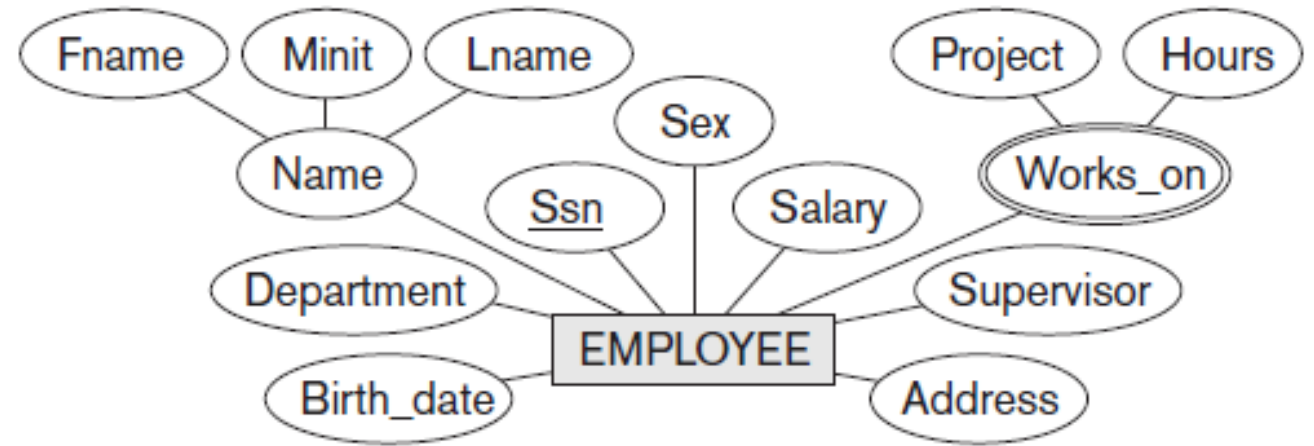
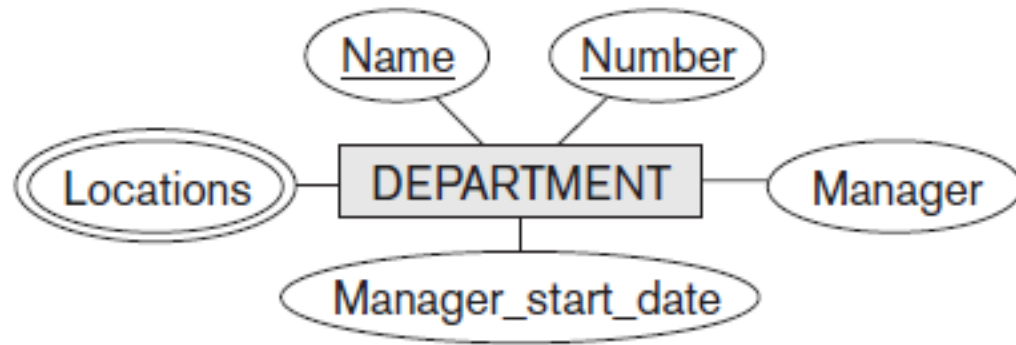
Figure 3.7

The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

Value Sets (Domains) of Attributes

- ✓ Each simple attribute of an entity type is associated with a **value set** (or domain of values), which specifies the set of values that may be assigned to that attribute for each individual entity
- ✓ if the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70
- ✓ Value sets are not typically displayed in basic ER diagrams and are similar to the basic data types available in most programming languages, such as integer, string, Boolean, float, enumerated type, subrange, and so on
- ✓ Mathematically, an attribute A of entity set E whose value set is V can be defined as a function from E to the power set P(V) of V: $A : E \rightarrow P(V)$
- ✓ We refer to the value of attribute A for entity e as A(e).
- ✓ The previous definition covers both single-valued and multivalued attributes, as well as NULLs.
- ✓ A NULL value is represented by the empty set
- ✓ For single-valued attributes, A(e) is restricted to being a singleton set for each entity e in E
- ✓ no restriction on multivalued attributes
- ✓ For a composite attribute A, the value set V is the power set of the Cartesian product of P(V1), P(V2), . . . , P(Vn), where V1, V2, . . . , Vn are the value sets of the simple component attributes that form A:
$$V = P(P(V1) \times P(V2) \times \dots \times P(Vn))$$

Initial Conceptual Design of the COMPANY Database



Relationship Types, Relationship Sets, Roles, and Structural Constraints

- Whenever an attribute of one entity type refers to another entity type, some relationship exists.
- For example:
 - the attribute Manager of DEPARTMENT refers to an employee who manages the department;
 - the attribute Controlling department of PROJECT refers to the department that controls the project;
 - the attribute Supervisor of EMPLOYEE refers to another employee (the one who supervises this employee);
 - the attribute Department of EMPLOYEE refers to the department for which the employee works; and so on.
- In the ER model, these references should not be represented as attributes but as **relationships**.

Relationship Types, Sets, and Instances

- A **relationship type** R among n entity types $E_1, E_2, \dots, E_n$ defines a set of associations—or a relationship set—among entities from these entity types.
- Similar to the case of entity types and entity sets, a relationship type and its corresponding relationship set are customarily referred to by the *same name*, R .

- Mathematically, the relationship set R is a set of **relationship instances** ri , where each ri associates n individual entities $(e_1, e_2, \dots, e_n)$, and each entity e_j in ri is a member of entity set E_j , $1 \leq j \leq n$.
- Hence, a relationship set is a mathematical relation on $E_1, E_2, \dots, E_n$;
- alternatively, it can be defined as a subset of the Cartesian product of the entity sets $E_1 \times E_2 \times \dots \times E_n$.
- Each of the entity types $E_1, E_2, \dots, E_n$ is said to **participate** in the relationship type R ;
- similarly, each of the individual entities $e_1, e_2, \dots, e_n$ is said to **participate** in the relationship instance $ri = (e_1, e_2, \dots, e_n)$.

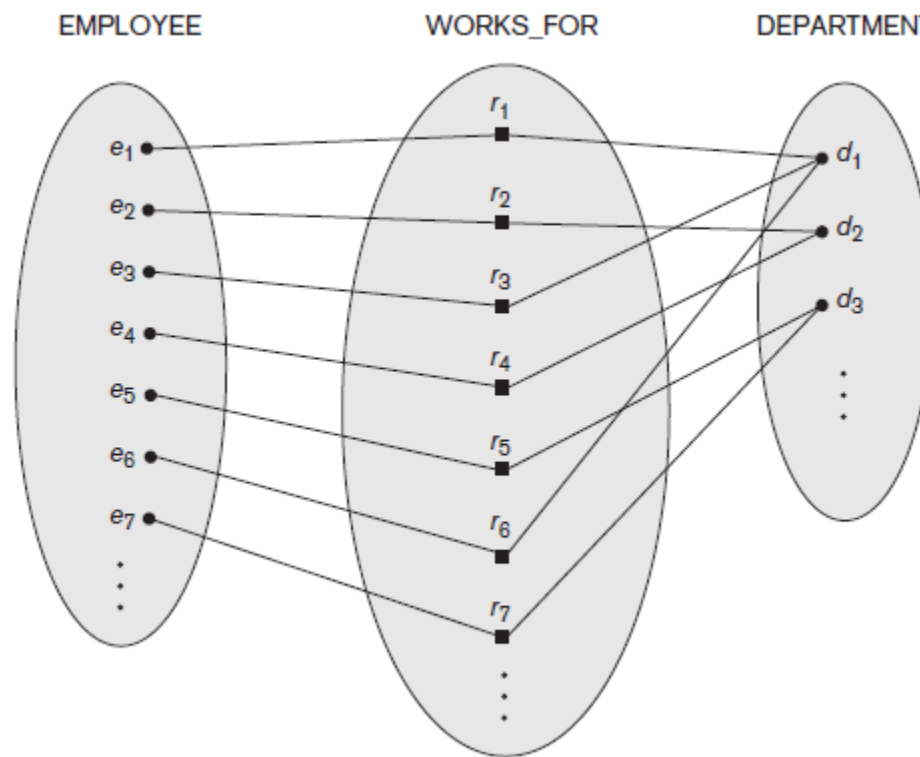
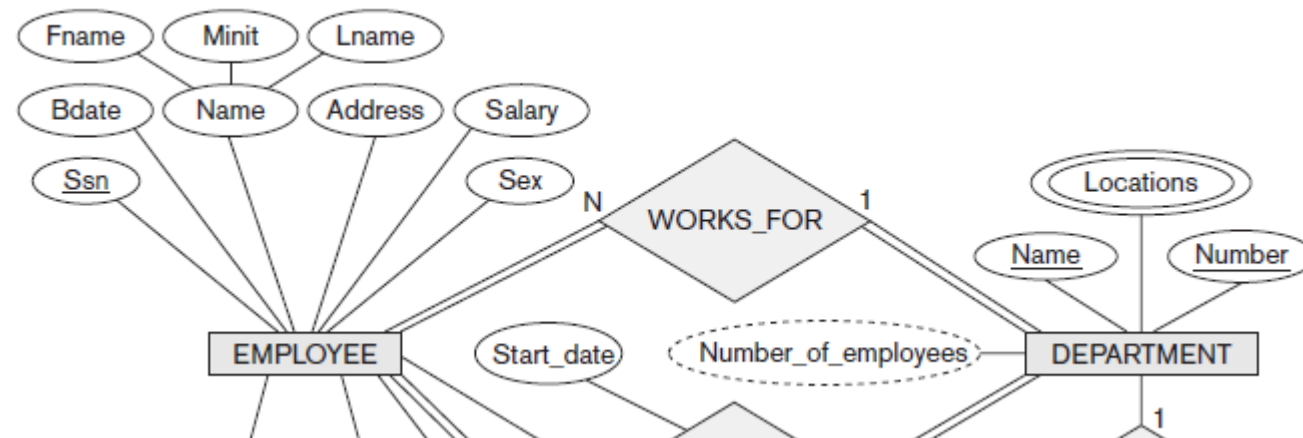


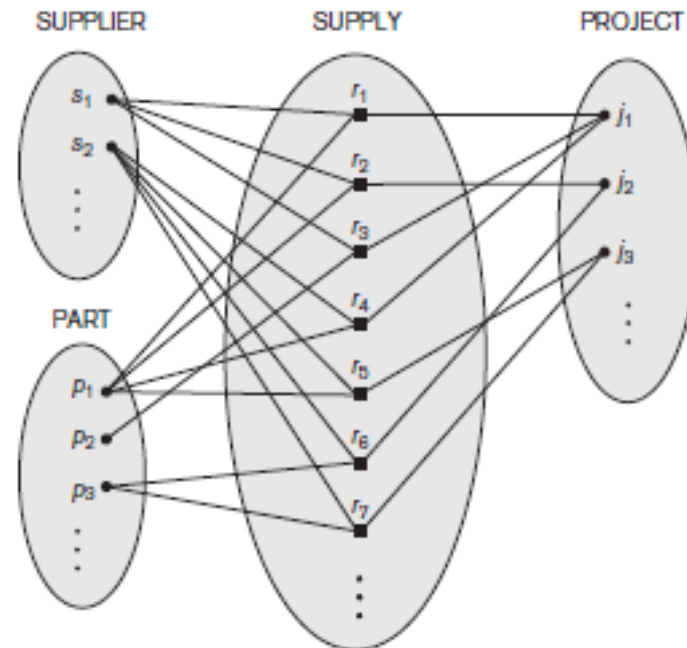
Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

- ✓ consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT, which associates each employee with the department for which the employee works. Each relationship instance in the relationship set WORKS_FOR associates one EMPLOYEE entity and one DEPARTMENT entity.
- ✓ the employees e1, e3, and e6 work for department d1
- ✓ the employees e2 and e4 work for department d2; and the employees e5 and e7 work for department d3
- ✓ In ER diagrams, relationship types are displayed as diamond-shaped boxes, which are connected by straight lines to the rectangular boxes representing the participating entity types. The relationship name is displayed in the diamond-shaped box



Relationship Degree, Role Names, and Recursive Relationships

- ✓ **Degree of a Relationship Type:** The degree of a relationship type is the number of participating entity types.
- ✓ Hence, the WORKS_FOR relationship is of degree two.
- ✓ A relationship type of degree two is called **binary**, and one of degree three is called **ternary**
- ✓ An example of a ternary relationship is SUPPLY, shown in Figure, where each relationship instance r_i associates three entities—a supplier s , a part p , and a project j —whenever s supplies part p to project j .



Role Names and Recursive Relationships

- ✓ Each entity type that participates in a relationship type plays a particular role in the relationship
 - ✓ the role name signifies the role that a participating entity from the entity type plays in each relationship instance
 - ✓ For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of employee or worker and DEPARTMENT plays the role of department or employer
 - ✓ same entity type participates more than once in a relationship type in different roles, such relationship types are called **recursive relationships** or **self-referencing relationships**.
- ✓ Below figure shows an example. The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set. Hence, the EMPLOYEE entity type *participates twice* in SUPERVISION: once in the role of *supervisor* (or *boss*), and once in the role of *supervisee* (or *subor*

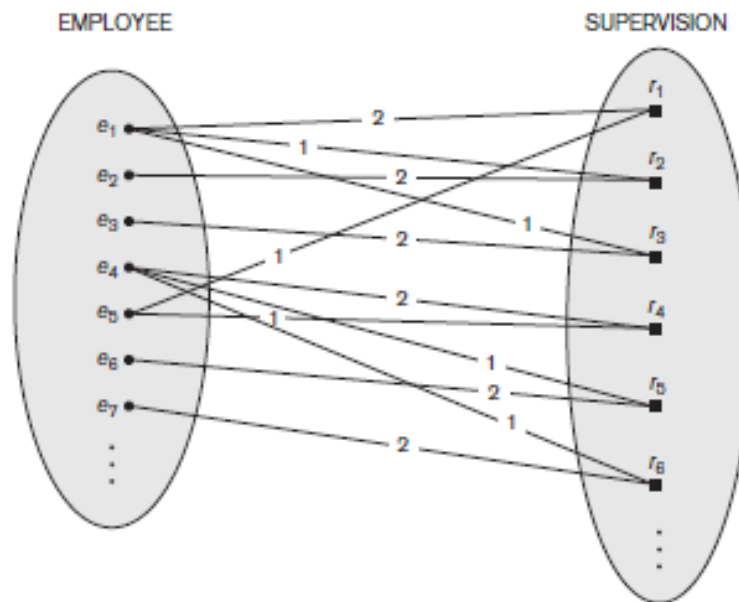


Figure 3.11
A recursive relationship
SUPERVISION
between EMPLOYEE
in the supervisor role
(1) and EMPLOYEE in
the subordinate role (2).

Constraints on Binary Relationship Types

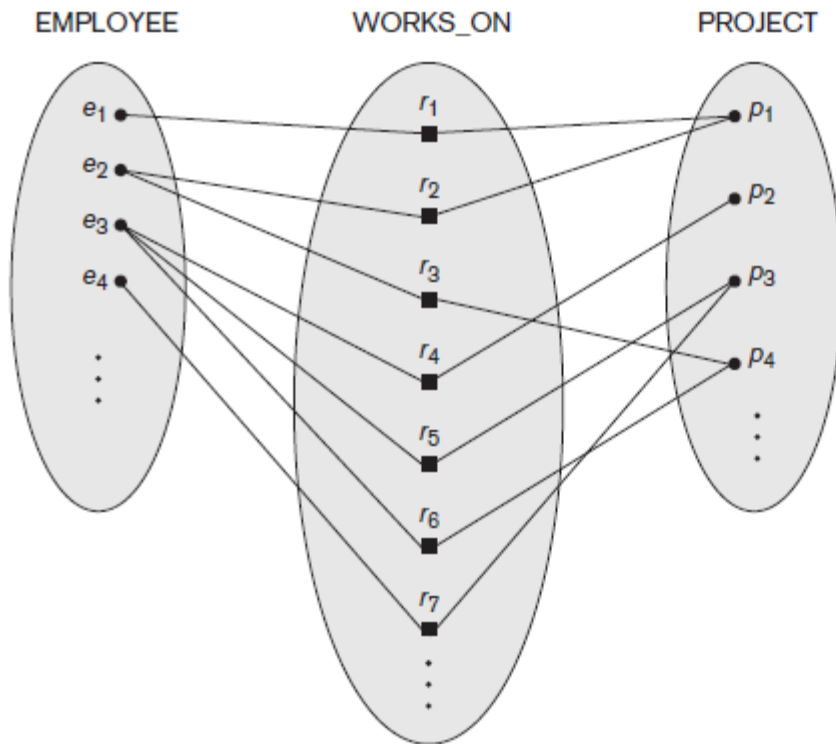
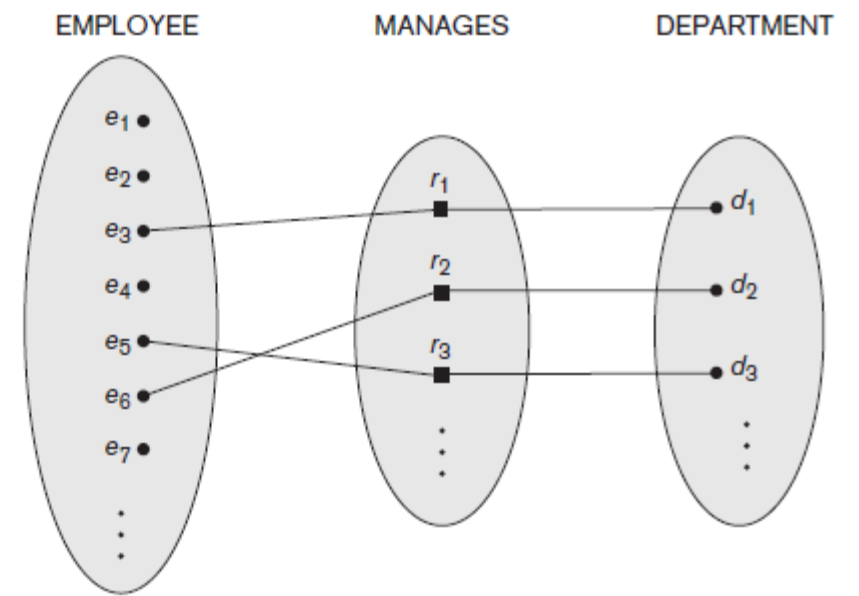
- Relationship types usually have certain constraints that limit the possible combinations of entities that may participate in the corresponding relationship set.
- These constraints are determined from the miniworld situation that the relationships represent.
- Two main types of binary relationship constraints: *cardinality ratio* and *participation*.

Cardinality Ratios for Binary Relationships.

- The cardinality ratio for a binary relationship specifies the **maximum number of relationship instances** that an entity can participate in.
- For example, in the WORKS_FOR binary relationship type, DEPARTMENT:EMPLOYEE is of cardinality ratio 1:N, meaning that each department can be related to (that is, employs) any number of employees (N), but an employee can be related to (work for) at most one department (1).
- This means that for this particular relationship type WORKS_FOR, a particular department entity can be related to any number of employees (N indicates there is no maximum number). On the other hand, an employee can be related to a maximum of one department.
- The possible cardinality ratios for binary relationship types are 1:1, 1:N, N:1, and M:N.

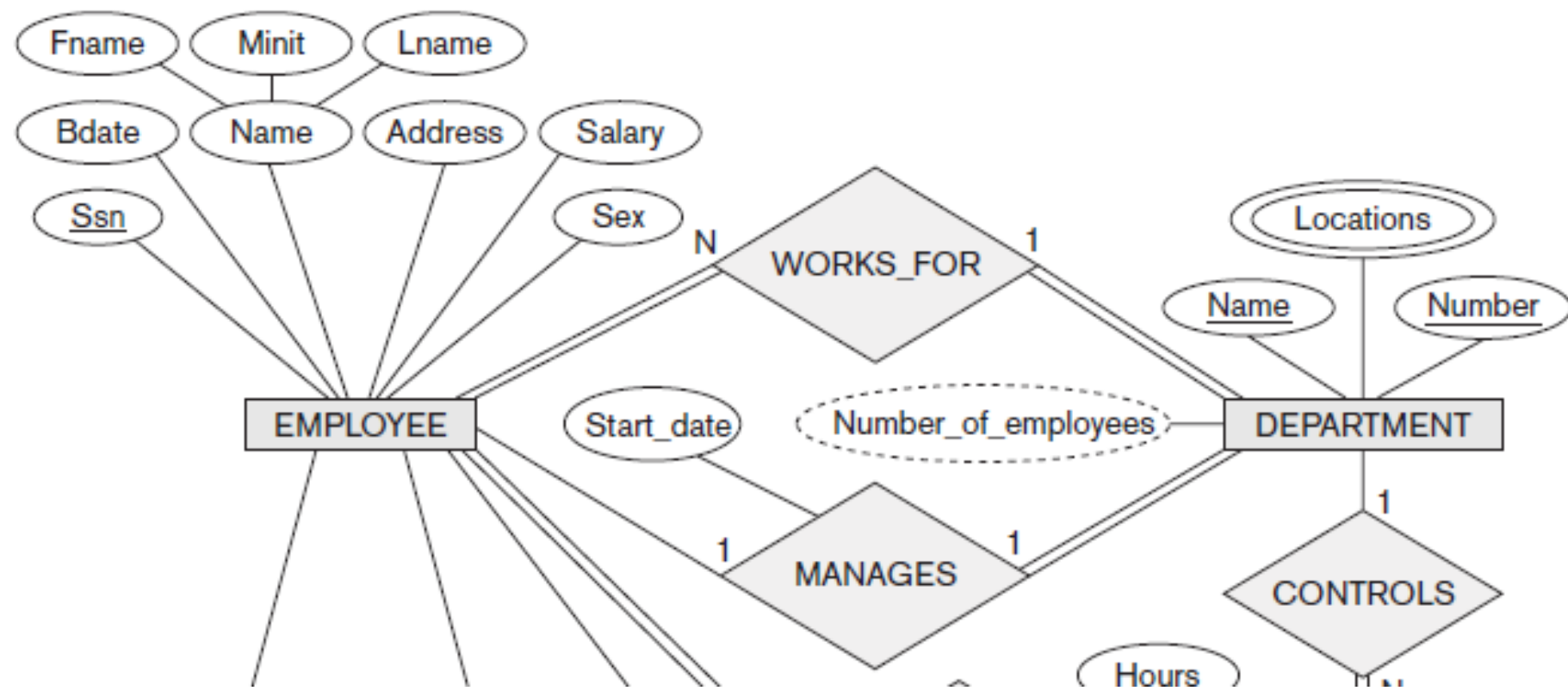
In 1:1, an employee can manage at most one department and a department can have at most one manager

Figure 3.12
A 1:1 relationship,
MANAGES.



In M:N, an employee can work on several projects and a project can have several employees

Figure 3.13
An M:N relationship,
WORKS_ON.

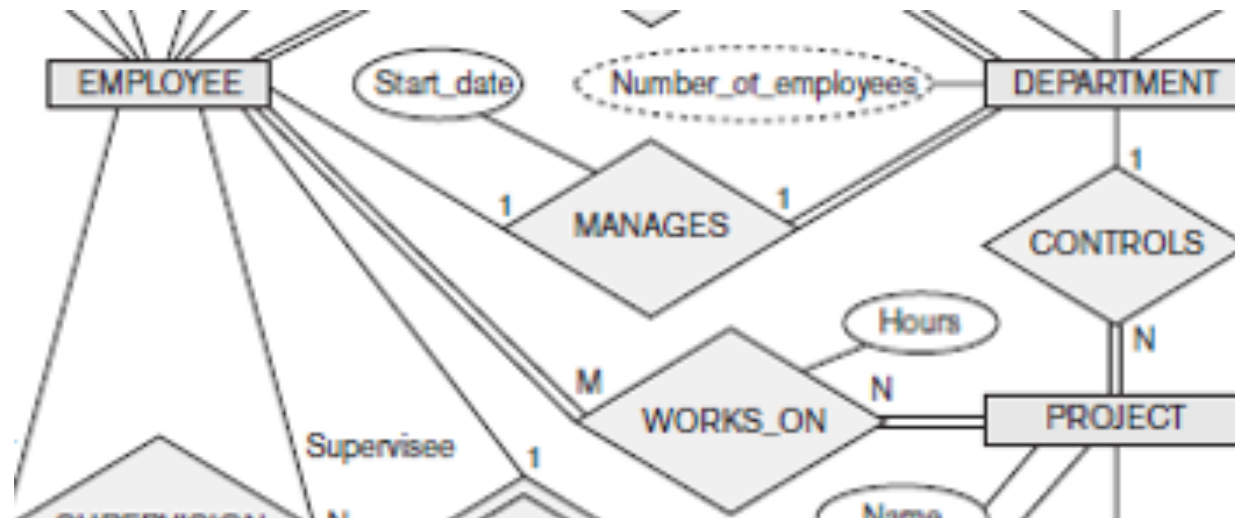


Participation Constraints and Existence Dependencies

- ✓ The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type
- ✓ This constraint specifies the minimum number of relationship instances that each entity can participate in and is sometimes called the minimum cardinality constraint.
- ✓ There are two types of participation constraints—**total and partial**
- ✓ If a company policy states that every employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance. Thus, the participation of EMPLOYEE in WORKS_FOR is called **total participation**, meaning that every entity in the total set of employee entities must be related to a department entity via WORKS_FOR. Total participation is also called **existence dependency**.
- ✓ we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial**, meaning that some or part of the set of employee entities are related to some department entity via MANAGES, but not necessarily all
- ✓ In ER diagrams, **total participation** (or existence dependency) is displayed as a **double line** connecting the participating entity type to the relationship, whereas **partial participation** is represented by a **single line**

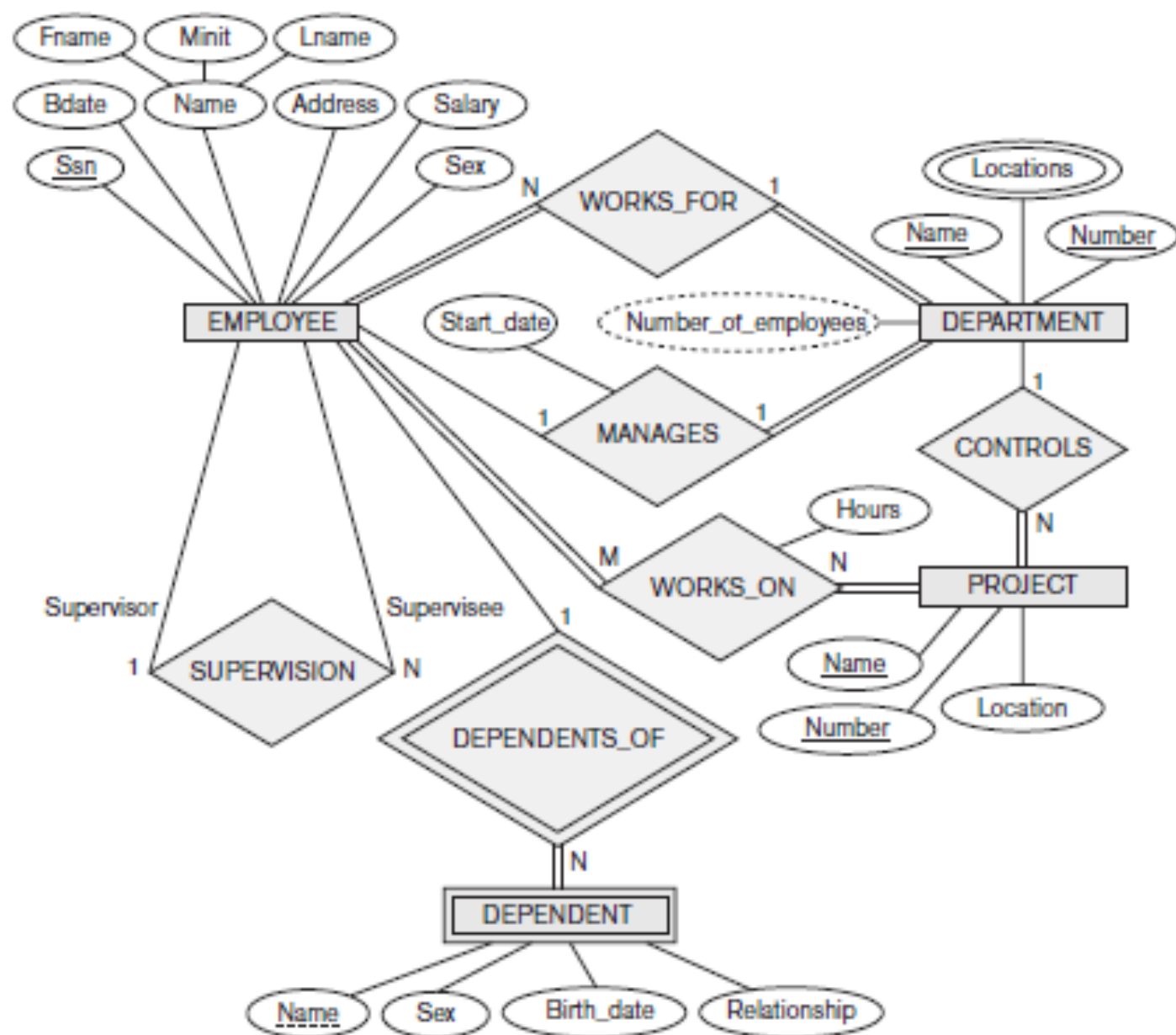
Attributes of Relationship Types

- ✓ Relationship types can also have attributes, similar to those of entity types.
- ✓ For example, to record the number of hours per week that a particular employee works on a particular project, we can include an attribute **Hours** for the **WORKS_ON** relationship type
- ✓ to include the date on which a manager started managing a department via an attribute **Start_date** for the **MANAGES** relationship type



Weak Entity Types

- ✓ Entity types that **do not have key attributes** of their own are called **weak entity types**.
- ✓ In contrast, **regular entity types** that do have a key attribute—are called **strong entity types**
- ✓ Entities belonging to a weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values.
- ✓ We call this other entity type the **identifying or owner entity type**, and we call the relationship type that relates a weak entity type to its owner the identifying relationship of the weak entity type
- ✓ A weak entity type always has a total participation constraint with respect to its identifying relationship because a weak entity cannot be identified without an owner entity
- ✓ A weak entity type normally has a **partial key**, which is the attribute that can **uniquely identify weak entities** that are related to the same owner entity
- ✓ assume that no two dependents of the same employee ever have the same first name, the attribute Name of DEPENDENT is the partial key.
- ✓ In ER diagrams, both a weak entity type and its identifying relationship are distinguished by surrounding their boxes and diamonds with double lines.
- ✓ The partial key attribute is underlined with a dashed or dotted line.



ER Diagrams, Naming Conventions, and Design Issues

Proper Naming of Schema Constructs

- ✓ Choose names that convey the meanings attached to the different constructs in the schema.
- ✓ Use singular names for entity types, rather than plural ones.
- ✓ Use the convention that entity type and relationship type names are in uppercase letters, attribute names have their initial letter capitalized, and role names are in lowercase letters.
- ✓ Nouns appearing in the narrative tend to give rise to entity type names, and the verbs tend to indicate names of relationship types.
- ✓ Choosing binary relationship names to make the ER diagram of the schema readable from left to right and from top to bottom.

Design Choices for ER Conceptual Design

Schema design process should be considered an **iterative refinement process**, where an initial design is created and then iteratively refined until the most suitable design is reached. Some of the refinements that are often used include the following:

- ✓ A concept may be first modeled as an attribute and then refined into a relationship because it is determined that the attribute is a reference to another entity type
- ✓ Similarly, an attribute that exists in several entity types may be elevated or promoted to an independent entity type. For example, suppose that each of several entity types in a UNIVERSITY database, such as STUDENT, INSTRUCTOR, and COURSE, has an attribute Department in the initial design; the designer may then choose to create an entity type DEPARTMENT with a single attribute Dept_name and relate it to the three entity types (STUDENT, INSTRUCTOR, and COURSE) via appropriate relationships
- ✓ An inverse refinement to the previous case may be applied—for example, if an entity type DEPARTMENT exists in the initial design with a single attribute Dept_name and is related to only one other entity type, STUDENT. In this case, DEPARTMENT may be reduced or demoted to an attribute of STUDENT

Alternative Notations for ER Diagrams

- We describe one alternative ER notation for specifying structural constraints on relationships, which replaces the cardinality ratio (1:1, 1:N, M:N) and single/double-line notation for participation constraints.
- This notation involves associating a pair of integer numbers (min, max) with each *participation* of an entity type E in a relationship type R , where $0 \leq \min \leq \max$ and $\max \geq 1$.
- The numbers mean that for each entity e in E , e must participate in at least *min* and at most *max* relationship instances in R *at any point in time*.
- In this method, $\min = 0$ implies partial participation, whereas $\min > 0$ implies total participation.
- Usually, one uses either the cardinality ratio/single-line/double-line notation *or* the (min, max) notation.

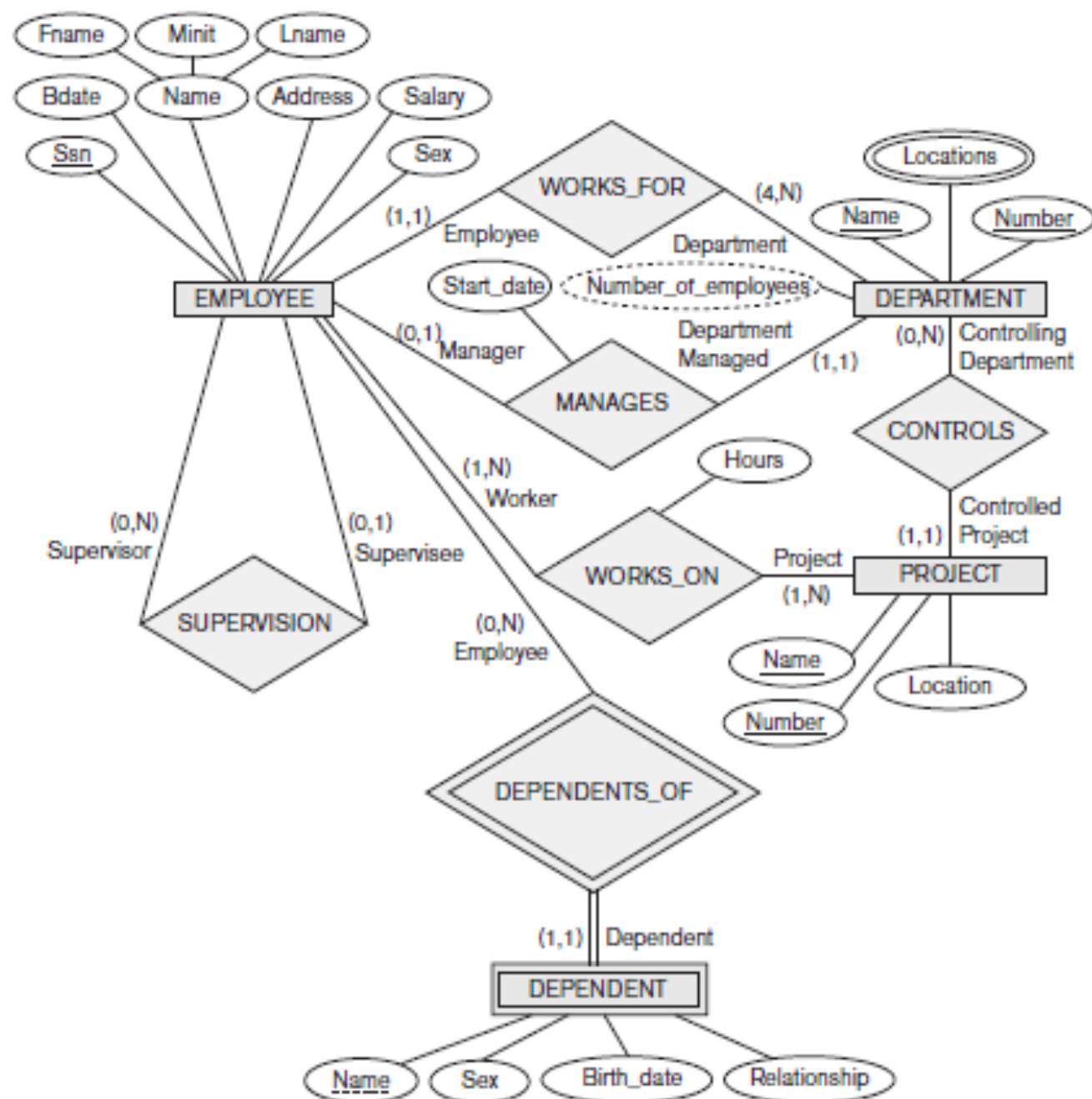
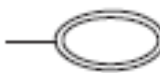
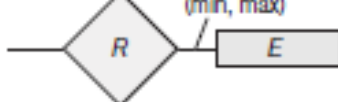


Figure 3.15

ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.

Summary of the notation for ER diagrams

Figure 3.14
Summary of the
notation for ER
diagrams.

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1 : N for $E_1 : E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Example of Other Notation:

UML Class Diagrams

The *entity types* are modeled as *Classes in UML*.

An *entity* in ER corresponds to an *object* in UML.

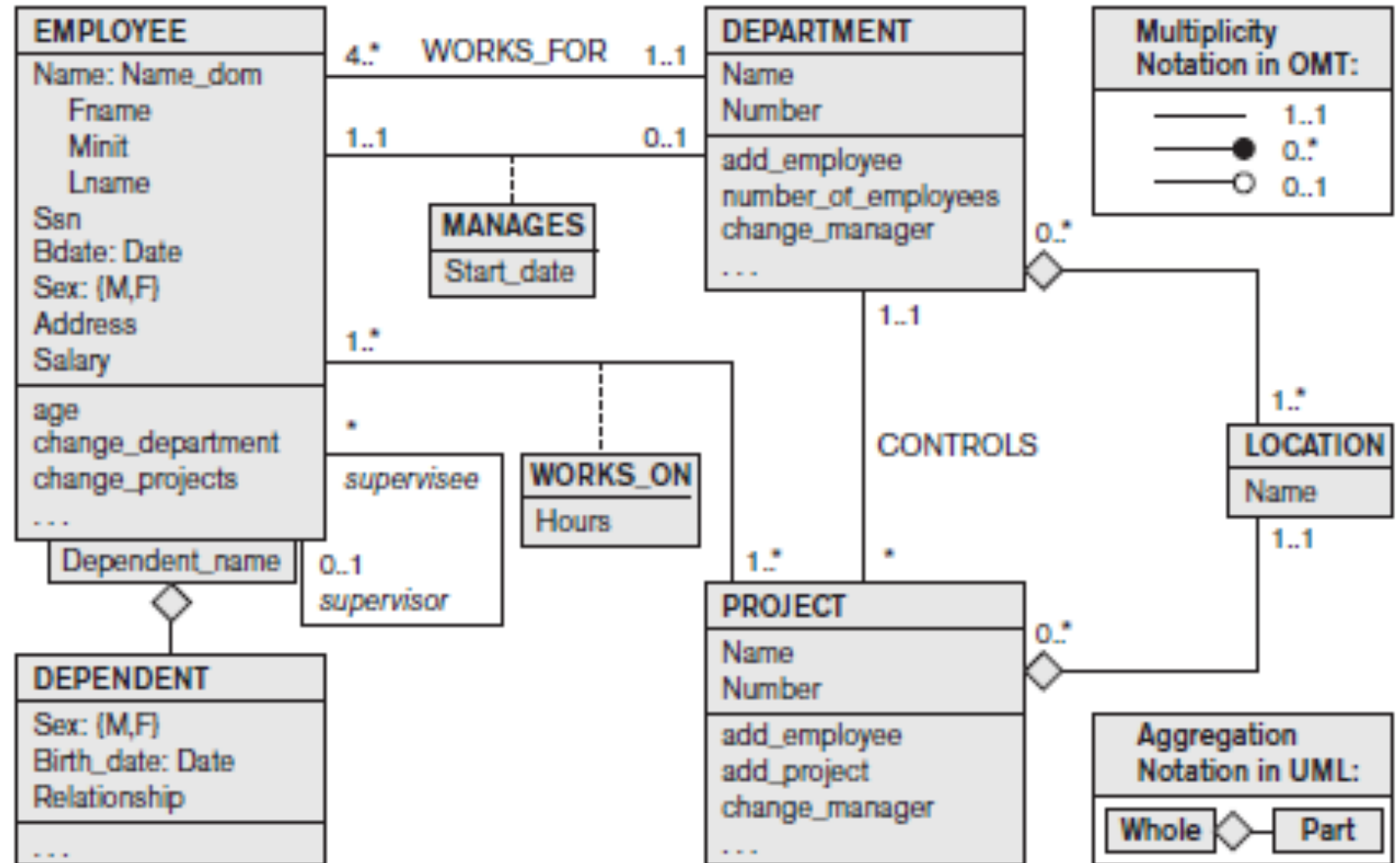


Figure 3.16

The COMPANY conceptual schema in UML class diagram notation.

In UML class diagrams, a **class** (similar to an entity type in ER) is displayed as a box that includes three sections:

- The top section gives the **class name** (similar to entity type name);
- the middle section includes the **attributes**;
- The last section includes **operations** that can be applied to individual objects (similar to individual entities in an entity set) of the class.

Operations are *not* specified in ER diagrams.

Relationship Types of Degree Higher than Two

Choosing between Binary and Ternary (or Higher-Degree) Relationships

- ✓ The ER diagram notation for a ternary relationship type is shown in Figure (a), which displays the schema for the SUPPLY relationship type that was displayed at the entity set/relationship set or instance level
- ✓ Recall that the relationship set of SUPPLY is a set of relationship instances (s, j, p) , where s is a SUPPLIER who is currently supplying a PART p to a PROJECT j
- ✓ In general, a relationship type R of degree n will have n edges in an ER diagram, one connecting R to each participating entity type.
- ✓ Figure (b) shows an ER diagram for three binary relationship types CAN_SUPPLY, USES, and SUPPLIES
- ✓ In general, a ternary relationship type represents different information than do three binary relationship types

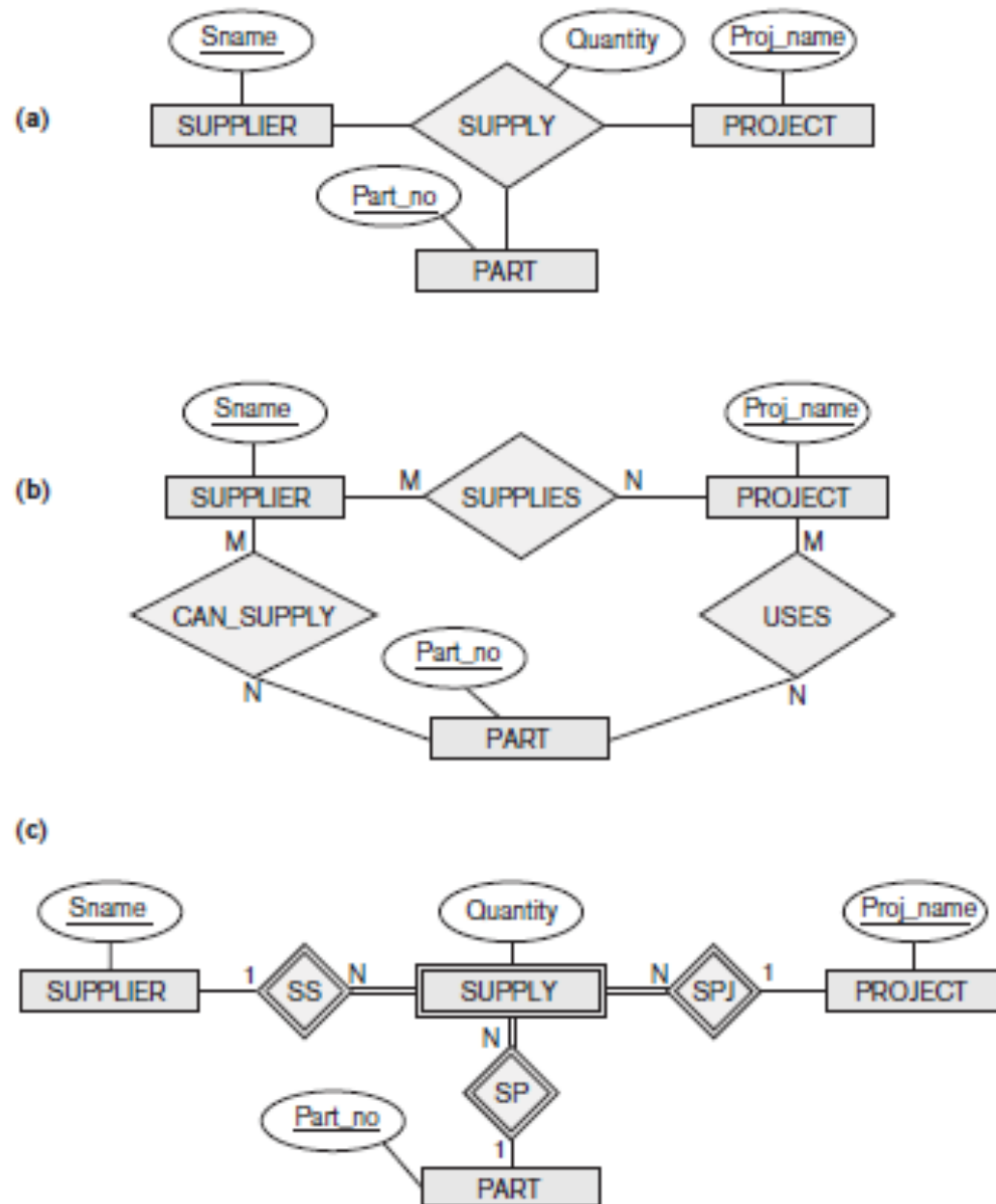


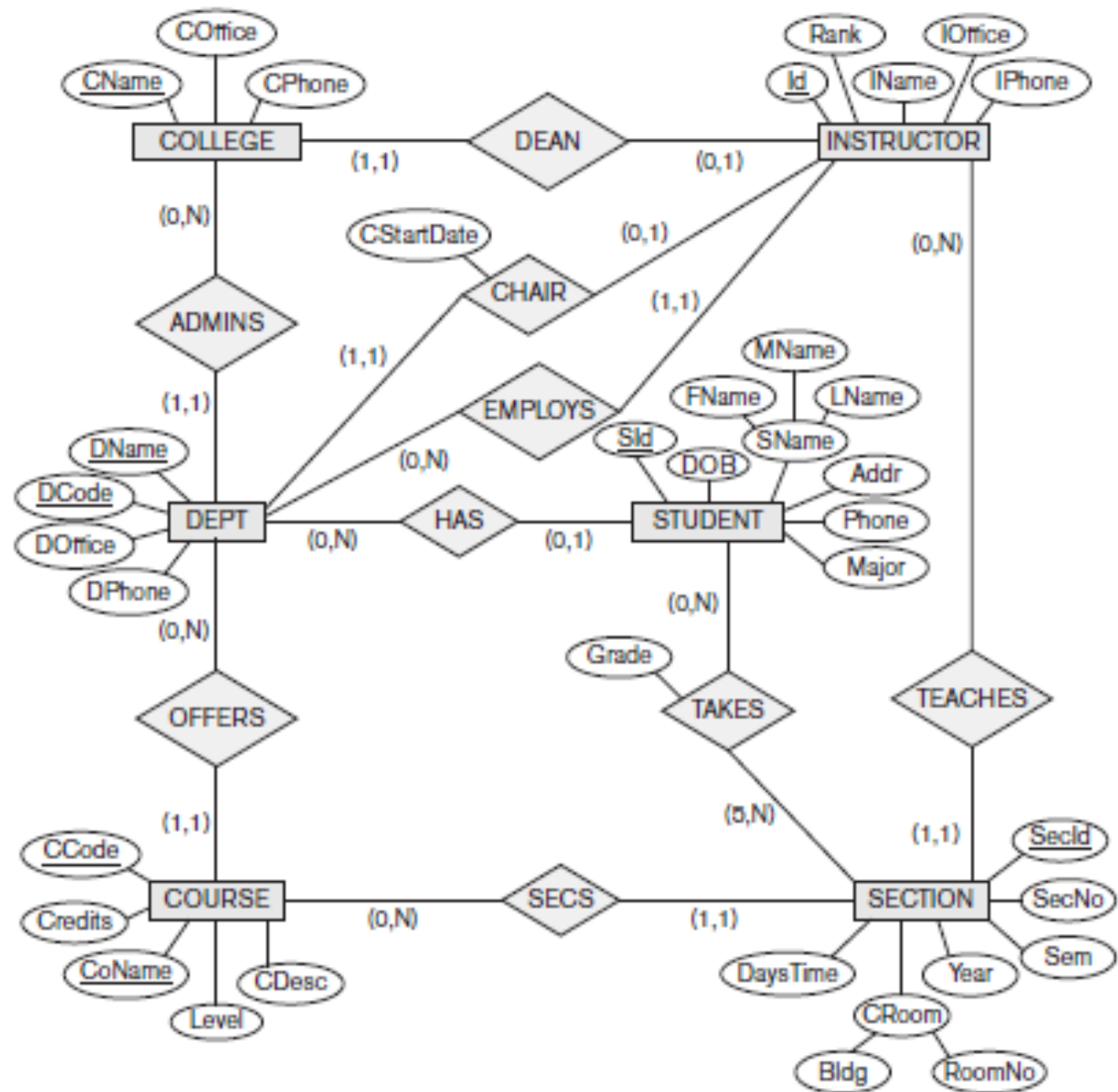
Figure 3.17

Ternary relationship types. (a) The SUPPLY relationship. (b) Three binary relationships not equivalent to SUPPLY. (c) SUPPLY represented as a weak entity type.

- ✓ Consider the three binary relationship types CAN_SUPPLY, USES, and SUPPLIES. Suppose that CAN_SUPPLY, between SUPPLIER and PART, includes an instance (s, p) whenever supplier s can supply part p (to any project); USES, between PROJECT and PART, includes an instance (j, p) whenever project j uses part p; and SUPPLIES, between SUPPLIER and PROJECT, includes an instance (s, j) whenever supplier s supplies some part to project j. The existence of three relationship instances (s, p), (j, p), and (s, j) in CAN_SUPPLY, USES, and SUPPLIES, respectively, does not necessarily imply that an instance (s, j, p) exists in the ternary relationship SUPPLY, because the meaning is different.
- ✓ It is often tricky to decide whether a particular relationship should be represented as a relationship type of degree n or should be broken down into several relationship types of smaller degrees
- ✓ The designer must base this decision on the semantics or meaning of the particular situation being represented

Some database design tools are based on variations of the ER model that permit only binary relationships. In this case, a ternary relationship such as SUPPLY must be represented as a weak entity type, with no partial key and with three identifying relationships. The three participating entity types SUPPLIER, PART, and PROJECT are together the owner entity types (see Figure (c)). Hence, an entity in the weak entity type SUPPLY in Figure (c) is identified by the combination of its three owner entities from SUPPLIER, PART, and PROJECT.

An ER diagram UNIVERSITY DB



An ER diagram for an AIRLINE DB

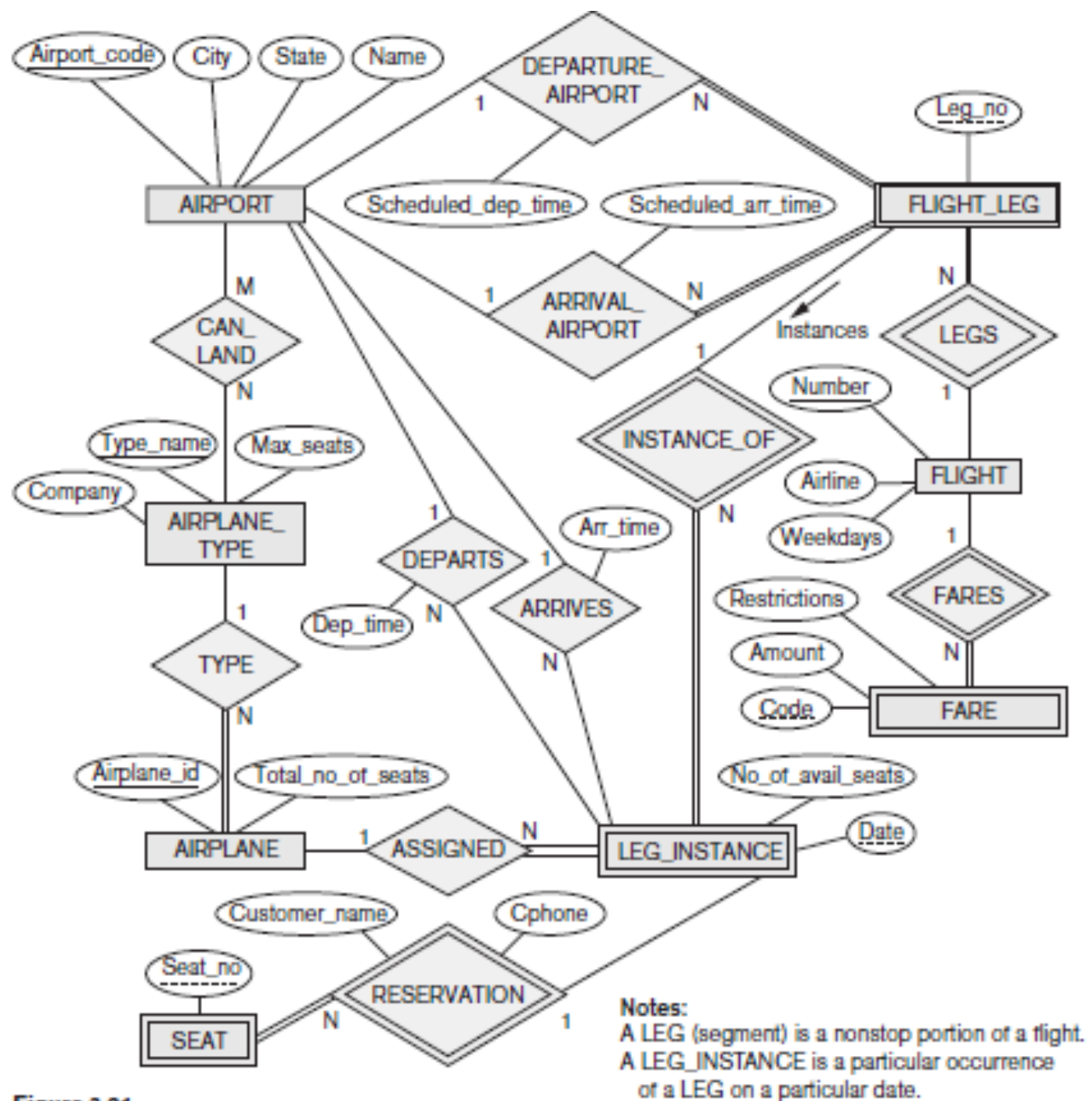


Figure 3.21

An ER diagram for BANK DB

